Reflection

CS-3180

Metaprogramming

- Treat other programs (functions, routines, etc) as data
- Macros + Decorators
- Last type we'll discuss in depth:
- Reflection

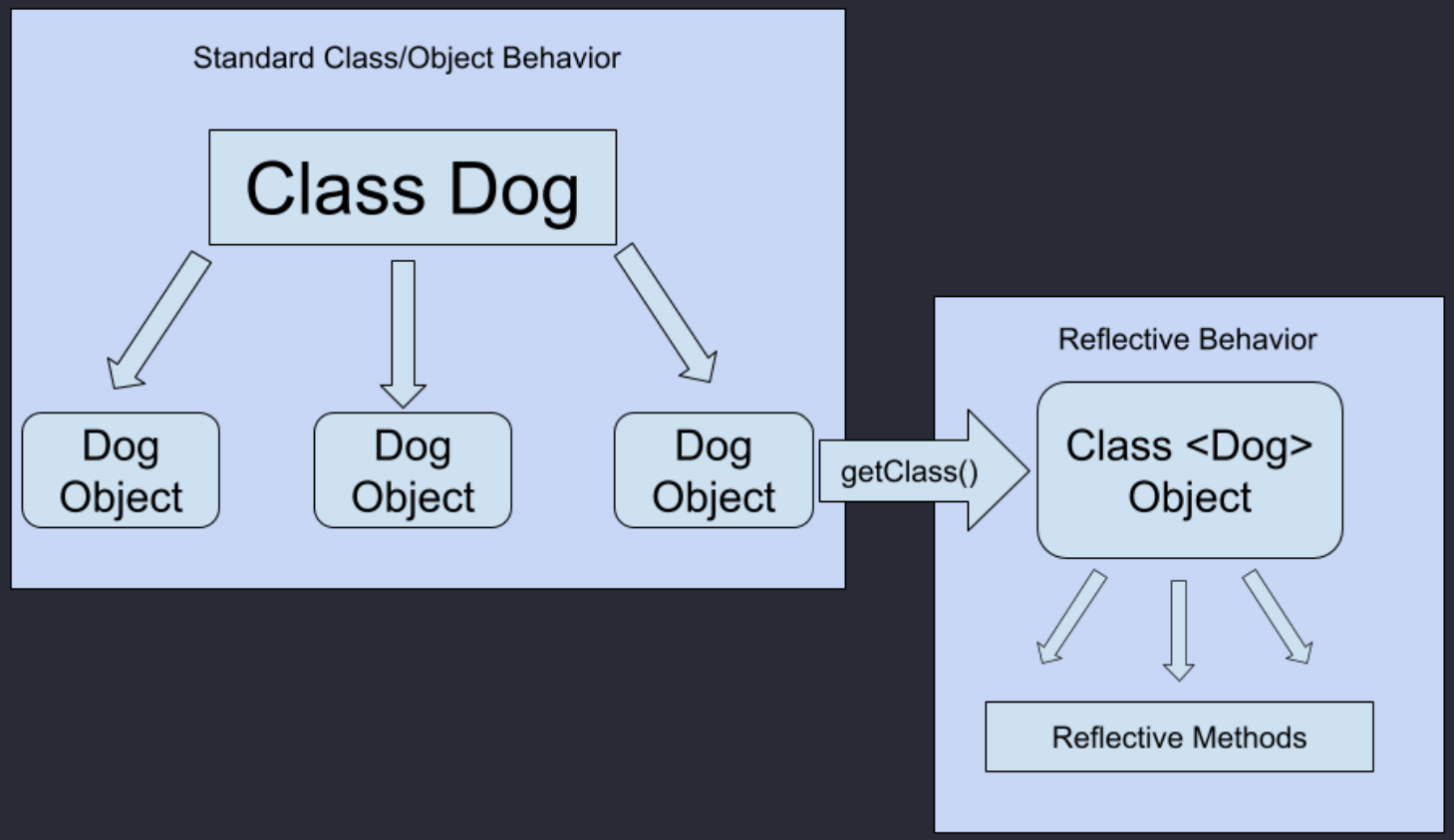
Reflection

- *The ability for a program to examine or modify aspects of itself at runtime*
- Introspect its own metadata
- E.g. what are the names of my fields, methods, decorators, etc
- Lisp is reflective by the nature of its homoiconicity

Java Reflection

- Java's standard library has `java.lang.reflect`
- Before we dive into this, need to get some vocabulary first
- Class is a blueprint for Object
- But Java also has Class Objects

- Calling `getClass()`
- Class Object who's type is Class you called it on
- Let's play around with this a bit



Reflection Space

- Once we have a class object, we are now in reflection space
- Enables you to see all of the things your LSP sees
- Use this as data, at runtime
- Let's see the things we have access to now

```
class Dog {... }

public static void main(String[] args) {
    Dog d = new Dog();
    // clazz is convention, since class is reserved
    Class<Dog> clazz = d.getClass();
}
```

Java Reflection

- `?` is the wildcard generic operator
- Mostly there because the `getClass()` method cannot be absolutely sure what it is an instance of
- (e.g. could be a subclass of `Dog`)
- "This is a Java type, but I don't know or care what type it is"

Dynamic Class Creation

- Using this, you can turn a string into a `Class`
- `Class.forName("Dog")`
- No instance has been created yet

```
class Dog {
    // ...
}

public class ReflectionTest {
    public static void main(String[] args) throws ClassNotFoundE
        String input = "Dog";
        Class<?> clazz = Class.forName(input);
        System.out.println(clazz);
    }
}
```

Flesh out our class

- To really show the power of reflection, our class needs more state
- Let's give it some:
- Private Fields
- Public and Private Methods
- (public) Constructors (for now)

Full Dog Class

- Mix of public/private methods/fields

```
1  class Dog {
2      public int ageInDogYears = 0;
3      private String name;
4      private Color color;
5
6      public Dog(String name, Color c){
7          this.ageInDogYears = 0;
8          this.name = name;
9          this.color = c;
10     }
11
12     // rich people do this right?
13     public Dog(Dog other){
14         this.ageInDogYears = other.ageInDogYears;
15         this.name = other.name;
16         this.color = c;
```

Full Dog Class

- Mix of public/private methods/fields

```
1  class Dog {
2      public int ageInDogYears = 0;
3      private String name;
4      private Color color;
5
6      public Dog(String name, Color c){
7          this.ageInDogYears = 0;
8          this.name = name;
9          this.color = c;
10     }
11
12     // rich people do this right?
13     public Dog(Dog other){
14         this.ageInDogYears = other.ageInDogYears;
15         this.name = other.name;
16         this.color = c;
```

Full Dog Class

- Mix of public/private methods/fields

```
12      // rich people do this right?
13      public Dog(Dog other){
14          this.ageInDogYears = other.ageInDogYears;
15          this.name = other.name;
16          this.color = c;
17      }
18
19      private void playFetch(){
20          System.out.println(this.name + "Plays fetch");
21      }
22
23      @Override
24      public String toString(){
25          return this.name + ", " + this.ageInDogYears + ", " +
26      }
27  }
```

Full Dog Class

- Mix of public/private methods/fields

```
1  class Dog {
2      public int ageInDogYears = 0;
3      private String name;
4      private Color color;
5
6      public Dog(String name, Color c){
7          this.ageInDogYears = 0;
8          this.name = name;
9          this.color = c;
10     }
11
12     // rich people do this right?
13     public Dog(Dog other){
14         this.ageInDogYears = other.ageInDogYears;
15         this.name = other.name;
16         this.color = c;
```

Dynamic Instantiation

- Instantiate a class via a `Constructor<?>`
- Specify which constructor we want
- Dyn. Dispatch will still call the correct `toString()`

```
1  Class<?> clazz = Class.forName("Dog");
2  Constructor<?> constr = clazz.getDeclaredConstructor(
3      String.class,
4      Color.class
5  );
6
7  Object dog = constr.newInstance(
8      "Clifford",
9      Color.RED
10 );
11 System.out.println(dog);
```

Dynamic Type Checking

- Consider what we are really doing here
- We have shifted our type errors
- From compile-time to run-time
- Deferred verification to runtime, making this code *dynamically-typed*
- In an otherwise statically typed language
- Enables all the benefits and flexibility of a dynamically-typed language in Java

Dynamic Field Access

- Access fields dynamically via `getDeclaredField(s)()`
- `clazz` and `instance` interact with each other

```
1  Class<?> clazz = Class.forName("Dog");
2  Constructor<?> constructor = clazz.getDeclaredConstructor(
3      String.class,
4      Color.class
5  );
6
7  Object instance = constructor.newInstance(
8      "Clifford",
9      Color.RED
10 );
11
12 Field f = clazz.getDeclaredField("ageInDogYears");
```

Dynamic Method Access

- Use `getMethod` to access a method from a string
- `invoke` to call it, with arguments if needed

```
1  Class<?> clazz = Class.forName("Dog");
2  Constructor<?> constructor = clazz.getDeclaredConstructor(
3      String.class,
4      Color.class
5  );
6
7  Object instance = constructor.newInstance(
8      "Clifford", Color.RED);
9
10 Method m = clazz.getMethod("toString"); // pass argument type
11 String output = (String) m.invoke(instance); // pass argument
12 System.out.println(output);
```

Encapsulation

- What happens if we try this for a private field?
- Well the data is encapsulated, but when we look at *where* the runtime error occurs...
- We can still create the Field/Method object
- We just can't access it
- Let's "fix" that!

Access Modifiers

- Fields/Method have a `setAccessible` method on them!
- Yikes!

```
1  Class<?> clazz = Class.forName("Dog");
2  Constructor<?> constructor = clazz.getDeclaredConstructor(
3      String.class,
4      Color.class
5  );
6
7  Object instance = constructor.newInstance("Clifford", Color.R
8
9  Field f = clazz.getDeclaredField("color"); // pass argument t
10 f.setAccessible(true);
11 Color c = (Color) f.get(instance);
12 System.out.println(c);
```

Breaking Encapsulation

- There are actually some classes in the std. library
- That *require* you to change an access modifier to use it
- You can actually make constructors private!
- Prevent instantiation*
- *Without reflection
- Enter `sun.misc.unsafe`

Sun Misc Unsafe

- Old-School Java Package for low-level access
- Allocate memory
- This will likely not work in future JDK versions, need special setup in current JDK
- Easier to show than demo:

```
1 Field f = Unsafe.class.getDeclaredField("theUnsafe");
2 f.setAccessible(true);
3 unsafe = (Unsafe) f.get(null);
```

https://www.baeldung.com/java-unsafe

Other Languages

- Reflection is not limited to Java
- Dynamically typed languages like JS and Python have similar mechanisms
- (This is what we were doing when we were accessing methods/fields of a Python class via a string)
- Technically, this is all *run-time* reflection
- Zig (+others) have a much stronger form called *compile-time* reflection

Aside: Is Java Homoiconic?

- Homoiconicity: code and data share the same representation
- Fields are data and can be treated like such!
- Consensus is actually *NO*
- Since ALL of the code (e.g. individual statements) cannot be represented

```
https://wiki.c2.com/?HomoiconicExampleInJava
```

Summary

- Reflection is a metaprogramming technique
- Allows for programs to introspect their own components
- In OOP languages, this means accessing fields and methods
- Can be done dynamically, at runtime

Questions?